

Lab2_FFNN_RNN_GNN

April 25, 2026

1 Laboratory 2 — Model Engineering

text

1.1 Setup

```
[ ]: # --- Check Python and pip versions ---  
!python --version  
!pip install --upgrade pip
```

Python 3.12.13

Requirement already satisfied: pip in /usr/local/lib/python3.12/dist-packages (26.0.1)

```
[ ]: # --- Install required libraries ---  
!pip install torch  
!pip install numpy pandas scikit-learn matplotlib seaborn  
!pip install tqdm  
!pip install torch_geometric  
# torch_sparse torch_scatter # TODO: check
```

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)

Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.5.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-

packages (0.13.2)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
 packages (from pandas) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
 packages (from pandas) (2025.3)
 Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
 packages (from scikit-learn) (1.16.3)
 Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
 packages (from scikit-learn) (1.5.3)
 Requirement already satisfied: threadpoolctl>=3.1.0 in
 /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
 Requirement already satisfied: contourpy>=1.0.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
 Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
 (4.67.3)
 Requirement already satisfied: torch_geometric in
 /usr/local/lib/python3.12/dist-packages (2.7.0)
 Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (3.13.4)
 Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (2025.3.0)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (3.1.6)
 Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
 (from torch_geometric) (2.0.2)
 Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (5.9.5)
 Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (3.3.2)
 Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-
 packages (from torch_geometric) (2.32.4)
 Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages

```
(from torch_geometric) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (3.6.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-
packages (from aiohttp->torch_geometric) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (0.4.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.23.0)
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-
packages (from yarll<2.0,>=1.17.0->aiohttp->torch_geometric) (3.11)
Requirement already satisfied: typing-extensions>=4.2 in
/usr/local/lib/python3.12/dist-packages (from
aiosignal>=1.4.0->aiohttp->torch_geometric) (4.15.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch_geometric) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (3.4.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric)
(2026.2.25)
```

```
[ ]: # --- Import libraries ---
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import math

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
↳LabelEncoder
```



```

=====|
| 0 Tesla T4 Off | 00000000:00:04.0 Off |
0 |
| N/A 77C P0 31W / 70W | 655MiB / 15360MiB | 0%
Default |
| | |
N/A |
+-----+-----+-----+
-----+

+-----+
-----+
| Processes:
|
| GPU GI CI PID Type Process name
GPU Memory |
| ID ID
Usage |
|=====|
=====|
| 0 N/A N/A 629 C /usr/bin/python3
652MiB |
+-----+
-----+

```

```

[ ]: # --- Check RAM availability ---
from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
    print('Not using a high-RAM runtime')
else:
    print('You are using a high-RAM runtime!')

```

Your runtime has 13.6 gigabytes of available RAM

Not using a high-RAM runtime

1.1.2 Paths setup

```

[ ]: # --- Mount Google Drive (for Google Colab users) ---
from google.colab import drive
drive.mount('/content/drive')

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ]: # --- Define Paths ---
group = 'AImSecure'
laboratory = 'Laboratory2'

base_path = '/content/drive/MyDrive/'
project_path = base_path + f'Projects/{group}/{laboratory}/'
data_path = project_path + 'data/'
results_path = project_path + 'results/'

# Ensure directories exist
os.makedirs(project_path, exist_ok=True)
os.makedirs(data_path, exist_ok=True)
os.makedirs(results_path, exist_ok=True)

print(f"Project path: {project_path}")
print(f>Data path: {data_path}")
print(f"Results path: {results_path}")
```

Project path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/
Data path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/data/
Results path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/

```
[ ]: # --- Set visual style ---
sns.set(style="whitegrid", palette="muted", font_scale=1.1)

def save_plot(fig: plt.Figure, filename: str, path: str = "./plots/", fmt: str =
    "png", dpi: int = 300, close_fig: bool = False) -> None:
    """
    Save a Matplotlib figure in a specific to a specified directory.

    Args:
        fig (plt.Figure): Matplotlib figure object to save.
        filename (str): Name of the file to save (e.g., 'plot.png').
        path (str, optional): Directory path to save the figure. Defaults to './
        plots/'.
        fmt (str, optional): File format for the saved figure. Defaults to
        'png'.
        dpi (int, optional): Dots per inch for the saved figure. Defaults to
        300.

    Returns:
        None
    """
    # Ensure the directory exists
    os.makedirs(path, exist_ok=True)
    save_path = os.path.join(path, f"{filename}.{fmt}")
```

```

# Save the figure
fig.savefig(save_path, bbox_inches='tight', pad_inches=0.1, dpi=dpi,
↪format=fmt)
# plt.close(fig) # Removed to display plots in notebook

if close_fig:
    plt.close(fig)

print(f"Saved plot: {save_path}")

```

```

[ ]: def _internal_plot_cm(y_true, y_pred, model_name, save_dir):
    """Internal helper to draw the confusion matrix plot."""
    cm = confusion_matrix(y_true, y_pred)

    # Create figure explicitly
    fig, ax = plt.subplots(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        cbar=False,
        xticklabels=['Benign (0)', 'Malware (1)'],
        yticklabels=['Benign (0)', 'Malware (1)'],
        ax=ax
    )

    ax.set_xlabel('Predicted Label', fontsize=12)
    ax.set_ylabel('True Label', fontsize=12)
    ax.set_title(f'Confusion Matrix: {model_name}', fontsize=14)

    if save_dir:
        clean_name = model_name.lower().replace(" ", "_").replace("-", "_")
        save_plot(
            fig=fig,
            filename=f"cm_{clean_name}",
            path=save_dir,
            fmt="png",
            dpi=300,
            close_fig=False
        )

    plt.show()

```


1.2 Task 1 — Frequency-based baseline

We implement a simple frequency-based baseline.

- Transform sequences into feature vectors counting API call occurrences.
- Helps evaluate whether simple approaches already perform well before using complex models.

```
[ ]: # Create directory for plots
save_dir = results_path + 'images/' + 'task1_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.2.1 Frequency-Based Approach

- Extract vocabulary from train and test datasets.
- Use vocabulary to create feature vectors (frequency counts per API call).
- Output dataframe: one row per sequence, one column per API call.

```
[ ]: # --- Load dataset and perform initial inspection ---

# Train set
file_path_train = data_path + 'train.json'
df_train = pd.read_json(file_path_train)

# Basic info
print("Train set:")
print("  Shape (raw):", df_train.shape)
print("  Columns:", list(df_train.columns))

# Test set
file_path_test = data_path + 'test.json'
df_test = pd.read_json(file_path_test)

# Basic info
print("\nTest set:")
print("  Shape (raw):", df_test.shape)
print("  Columns:", list(df_test.columns))
```

Train set:

Shape (raw): (16325, 2)
Columns: ['api_call_sequence', 'is_malware']

Test set:

Shape (raw): (6505, 2)
Columns: ['api_call_sequence', 'is_malware']

```
[ ]: df_train
```

```
[ ]:
          api_call_sequence  is_malware
0  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
```

2	[FindResourceExW, LoadResource, FindResourceEx...	1
3	[FindResourceExW, LoadResource, FindResourceEx...	1
4	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	1
...	...	...
16320	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	1
16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

[16325 rows x 2 columns]

```
[ ]: malware_counts = df_train['is_malware'].value_counts()
num_total_samples = len(df_train)

print("Malware vs. Benign Samples in Training Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f" Malware (1): {count} samples ({percentage:.2f}%) - {(count /
↳ num_total_samples)*100:.2f}% of total")
    else:
        print(f" Benign (0): {count} samples ({percentage:.2f}%) - {(count /
↳ num_total_samples)*100:.2f}% of total")
```

Malware vs. Benign Samples in Training Set:

is_malware

1 15714

0 611

Name: count, dtype: int64

Percentages:

Malware (1): 15714 samples (96.26%) - 96.26% of total

Benign (0): 611 samples (3.74%) - 3.74% of total

```
[ ]: df_test
```

	api_call_sequence	is_malware
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0

6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

[6505 rows x 2 columns]

```
[ ]: malware_counts = df_test['is_malware'].value_counts()
num_total_samples = len(df_test)

print("Malware vs. Benign Samples in Test Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f" Malware (1): {count} samples ({percentage:.2f}%) - {(count /
↪num_total_samples)*100:.2f}% of total")
    else:
        print(f" Benign (0): {count} samples ({percentage:.2f}%) - {(count /
↪num_total_samples)*100:.2f}% of total")
```

Malware vs. Benign Samples in Test Set:

is_malware

1 6262

0 243

Name: count, dtype: int64

Percentages:

Malware (1): 6262 samples (96.26%) - 96.26% of total

Benign (0): 243 samples (3.74%) - 3.74% of total

1.2.2 Extract the vocabularies

```
[ ]: # Extract vocabulary from training set
train_vocab = set(api_call for seq in df_train['api_call_sequence'] for
↪api_call in seq)

# Count unique API calls
print(f"Train: {len(train_vocab)}")
```

Train: 258

```
[ ]: # Extract vocabulary from test set
test_vocab = set(api_call for seq in df_test['api_call_sequence'] for api_call
↪in seq)
```

```
# Count unique API calls
print(f"Test: {len(test_vocab)}")
```

Test: 232

Q: How many unique API calls does the training set contain? How many in the test set?

- Training set: there are **258** unique API calls.
- Test set: there are **232** unique API calls.

```
[ ]: # --- Find API calls only in the training set ---

only_in_train = train_vocab - test_vocab

# Count and print them
print(f"Number of API calls only in training set: {len(only_in_train)}")
print("API calls only in training set:", only_in_train)
```

Number of API calls only in training set: 29
 API calls only in training set: {'RegDeleteKeyA', 'InternetReadFile', 'IWbemServices_ExecMethod', 'RtlRemoveVectoredExceptionHandler', 'CryptProtectMemory', 'CertOpenSystemStoreW', 'CertOpenStore', 'NtWriteVirtualMemory', 'GetAddrInfoW', 'sendto', 'NetUserGetInfo', 'CreateRemoteThread', 'ObtainUserAgentString', 'NtLoadKey', 'listen', 'GetUserNameExW', 'NtReadVirtualMemory', 'NetUserGetLocalGroups', 'GetFileInformationByHandleEx', 'InternetOpenUrlW', 'InternetSetStatusCallback', 'CryptDecodeObjectEx', 'recvfrom', 'GetUserNameExA', 'NtDeleteFile', 'GetKeyboardState', 'FindFirstFileExA', 'SendNotifyMessageW', 'WSASocketW'}

```
[ ]: # --- Find API calls only in the test set ---

only_in_test = test_vocab - train_vocab

# Count and print them
print(f"Number of API calls only in test set: {len(only_in_test)}")
print("API calls only in test set:", only_in_test)
```

Number of API calls only in test set: 3
 API calls only in test set: {'WSASocketA', 'NtDeleteKey', 'ControlService'}

```
[ ]: # Create a boolean series to identify samples in df_test that contain any of
      ↪ these API calls
has_test_only_apis = df_test['api_call_sequence'].apply(
    lambda seq: any(api in only_in_test for api in seq)
)

# Count the samples
samples_with_test_only_apis = has_test_only_apis.sum()
```

```
print(f"Number of test set samples with API calls unique to the test set:␣
↪{samples_with_test_only_apis}")

# Display the actual rows
print("\nTest samples containing API calls unique to the test set:")
display(df_test[has_test_only_apis])
```

Number of test set samples with API calls unique to the test set: 3

Test samples containing API calls unique to the test set:

	api_call_sequence	is_malware
2873	[LdrGetDllHandle, LdrGetProcedureAddress, LdrG...	1
4105	[NtProtectVirtualMemory, CreateThread, NtAlloc...	1
5948	[NtDelayExecution, gethostbyname, NtDelayExecu...	1

Q: Are there any API calls that appear only in the test set (but not in the training set)? If yes, how many? Which ones? Yes — there are API calls that appear only in the test set and not in the training set.

- Number of such API calls: 3
- API calls only in the test set: NtDeleteKey, ControlService, WSASocketA

1.2.3 New frequency-based dataframes

- Create dataframe using training vocabulary as features.
- Count occurrences of each API call per sequence.

```
[ ]: # Create frequency-based dataframe for training data
train_df_freq = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for␣
↪seq in df_train['api_call_sequence']])
```

```
[ ]: # Dataframe characteristics
print("Shape (raw):", train_df_freq.shape)
print("Columns:", list(train_df_freq.columns))
```

Shape (raw): (16325, 258)

Columns: ['StartServiceA', 'NtGetContextThread', 'IWbemServices_ExecQuery', 'FindResourceA', 'LdrGetProcedureAddress', 'InternetGetConnectedState', 'NtOpenProcess', 'GetNativeSystemInfo', 'GetAddrInfoW', 'CreateToolhelp32Snapshot', 'ShellExecuteExW', 'InternetCrackUrlW', 'CryptExportKey', 'WriteProcessMemory', 'NtCreateFile', 'GlobalMemoryStatus', 'EnumServicesStatusA', 'LdrLoadDll', 'OpenServiceA', 'ReadProcessMemory', 'LookupPrivilegeValueW', 'NtFreeVirtualMemory', 'GetComputerNameA', 'SetFileTime', 'RegDeleteKeyW', 'IsDebuggerPresent', 'LdrUnloadDll', 'OpenSCManagerW', 'GetFileInformationByHandle', 'connect', 'CryptAcquireContextW', 'NtSetValueKey', 'GetSystemTimeAsFileTime', 'InternetConnectA', 'RegSetValueExA', 'DeviceIoControl',

'GetFileInformationByHandleEx', 'OpenSCManagerA', 'recvfrom', 'FindResourceW',
 'GetSystemDirectoryA', 'RegCreateKeyExW', 'GetUserNameExA',
 'NtOpenDirectoryObject', 'NtQueryValueKey', 'GetKeyboardState',
 'NtQuerySystemInformation', 'GetFileSize', 'WSAStartup', 'RegDeleteValueW',
 'FindFirstFileExA', 'SHGetFolderPathW', 'WSARecv', 'GetSystemMetrics',
 'SendNotifyMessageW', 'OleInitialize', 'CryptHashData', 'CreateServiceW',
 'NtSetContextThread', 'WSASocketW', 'HttpOpenRequestW', 'RegDeleteKeyA',
 'LdrGetDllHandle', 'GetAsyncKeyState', 'SetUnhandledExceptionFilter',
 'RtlAddVectoredExceptionHandler', 'timeGetTime', 'NtCreateMutant',
 'RegQueryInfoKeyW', 'GetForegroundWindow', 'FindWindowW', 'FindResourceExW',
 'RegOpenKeyExA', 'NtReadFile', 'NtEnumerateValueKey', 'RegEnumValueW',
 'CoCreateInstance', 'DnsQuery_A', 'NtWriteVirtualMemory', 'InternetOpenW',
 'RemoveDirectoryW', 'CreateJobObjectW', 'socket', 'GetFileAttributesW',
 'select', 'InternetQueryOptionA', 'GetFileSizeEx', 'LoadStringA', 'shutdown',
 'CryptDecrypt', 'CryptGenKey', 'CopyFileExW', 'NtLoadKey', 'InternetOpenUrlA',
 'CoGetClassObject', 'SetFilePointerEx', 'DeleteFileW', 'GlobalMemoryStatusEx',
 'WriteConsoleA', 'GetTimeZoneInformation', 'GetVolumeNameForVolumeMountPointW',
 'RtlDecompressBuffer', 'SetFilePointer', 'NetUserGetLocalGroups', 'RegEnumKeyW',
 'CopyFileW', 'RegEnumKeyExA', 'InternetOpenUrlW', 'InternetSetStatusCallback',
 'CoUninitialize', 'GetAdaptersAddresses', 'RegCloseKey', 'HttpSendRequestA',
 'RegEnumKeyExW', 'GetSystemInfo', 'GetBestInterfaceEx', 'getaddrinfo',
 'RegOpenKeyExW', 'RegCreateKeyExA', 'NtOpenMutant', 'SetErrorMode',
 'CoCreateInstanceEx', 'GetFileType', 'NtOpenFile', 'NtMapViewOfSection',
 'IWbemServices_ExecMethod', 'RtlRemoveVectoredExceptionHandler',
 'GetDiskFreeSpaceExW', 'NtAllocateVirtualMemory', 'NtClose', 'SizeofResource',
 'GetFileVersionInfoExW', 'CoInitializeSecurity', 'GetKeyState', 'GetUserNameW',
 'CryptProtectMemory', 'GetFileVersionInfoW', 'GetComputerNameW',
 'CertOpenStore', 'StartServiceW', 'GetFileVersionInfoSizeExW', 'NtOpenKeyEx',
 'NtOpenSection', 'NtSetInformationFile', 'SearchPathW', 'CreateDirectoryW',
 'InternetSetOptionA', 'LookupAccountSidW', 'GetCursorPos', 'NtTerminateThread',
 'sendto', 'getsockname', 'NtQueryInformationFile', 'WriteConsoleW',
 'InternetCloseHandle', 'GetSystemWindowsDirectoryW', 'Process32NextW',
 'CreateThread', 'RegDeleteValueA', 'recv', 'NtQueryAttributesFile',
 'NtUnmapViewOfSection', 'GetFileAttributesExW', 'RegSetValueExW',
 'Thread32Next', 'listen', 'NtOpenKey', 'CryptDecodeObjectEx',
 'GetDiskFreeSpaceW', 'NtDeleteFile', 'EnumWindows', '__exception__',
 'Module32FirstW', 'GetVolumePathNameW', 'NtQueryDirectoryFile', 'DrawTextExA',
 'CryptProtectData', 'CryptCreateHash', 'UnhookWindowsHookEx', 'CreateServiceA',
 'OpenServiceW', 'NtProtectVirtualMemory', 'GetAdaptersInfo', 'RemoveDirectoryA',
 'HttpQueryInfoA', 'Thread32First', 'DrawTextExW', 'InternetReadFile',
 'setsockopt', 'GetVolumePathNamesForVolumeNameW', 'RegQueryInfoKeyA',
 'CryptEncrypt', 'NtOpenThread', 'InternetCrackUrlA', 'CopyFileA',
 'NtQueueApcThread', 'CertOpenSystemStoreW', 'NtCreateSection',
 'DeleteUrlCacheEntryA', 'LoadStringW', 'RegQueryValueExW',
 'GetSystemDirectoryW', 'RegisterHotKey', 'GetFileVersionInfoSizeW',
 'NtWriteFile', 'Module32NextW', 'FindFirstFileExW', 'UuidCreate',
 'GetUserNameA', 'NtQueryKey', 'CryptAcquireContextA', 'FindWindowA', 'bind',
 'ioctlsocket', 'CreateActCtxW', 'NtTerminateProcess', 'SetWindowsHookExA',

```
'NtDeviceIoControlFile', 'CreateProcessInternalW', 'NtDuplicateObject',
'CreateRemoteThread', 'NetUserGetInfo', 'FindWindowExA', 'SetStdHandle',
'ObtainUserAgentString', 'NtEnumerateKey', 'RegEnumValueA', 'NtResumeThread',
'HttpOpenRequestA', 'send', 'SetFileAttributesW', 'NtDelayExecution',
'Process32FirstW', 'SHGetSpecialFolderLocation', 'GetUserNameExW',
'NtReadVirtualMemory', 'MessageBoxTimeoutA', 'NtCreateKey', 'closesocket',
'SetWindowsHookExW', 'InternetOpenA', 'FindWindowExW', 'NtSuspendThread',
'MoveFileWithProgressW', 'GetSystemWindowsDirectoryA', 'NtCreateThreadEx',
'DeleteUrlCacheEntryW', 'MessageBoxTimeoutW', 'OutputDebugStringA',
'gethostbyname', 'SetEndOfFile', 'InternetConnectW', 'LoadResource',
'CoInitializeEx', 'RegQueryValueExA', 'GetTempPathW', 'GetShortPathNameW',
'FindResourceExA']
```

```
[ ]: train_df_freq.head(10)
```

```
[ ]:      StartServiceA  NtGetContextThread  IWbemServices_ExecQuery  FindResourceA  \
0                0                0                0                0
1                0                0                0                0
2                0                0                0                0
3                0                0                0                1
4                0                0                0                0
5                0                0                0                1
6                0                0                0                0
7                0                0                0                0
8                0                0                0                1
9                0                0                0                0
```

```
      LdrGetProcedureAddress  InternetGetConnectedState  NtOpenProcess  \
0                16                0                0
1                10                0                0
2                10                0                0
3                 6                0                0
4                10                0                0
5                 6                0                0
6                35                0                1
7                 5                0                0
8                 6                0                0
9                 5                0                0
```

```
      GetNativeSystemInfo  GetAddrInfoW  CreateToolhelp32Snapshot  ...  \
0                0                0                0  ...
1                0                0                0  ...
2                0                0                0  ...
3                0                0                0  ...
4                0                0                0  ...
5                0                0                0  ...
6                0                0                0  ...
```

7	0	0	0	...
8	0	0	0	...
9	0	0	0	...

	OutputDebugStringA	gethostbyname	SetEndOfFile	InternetConnectW	\
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0
6	0	0	0	0	0
7	0	0	0	0	0
8	0	0	0	0	0
9	0	0	0	0	0

	LoadResource	CoInitializeEx	RegQueryValueExA	GetTempPathW	\
0	8	0	0	0	0
1	0	0	0	0	0
2	21	0	0	0	0
3	18	0	0	0	0
4	16	0	0	0	0
5	23	0	0	0	0
6	0	0	2	0	0
7	9	0	0	0	0
8	13	0	0	0	0
9	10	0	0	0	0

	GetShortPathNameW	FindResourceExA
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	0

[10 rows x 258 columns]

```
[ ]: # Create frequency-based dataframe for test data
test_df_freq = pd.DataFrame([api: seq.count(api) for api in test_vocab] for
    seq in df_test['api_call_sequence'])
```



```
[ ]: # Dataframe characteristics
print("Shape (raw):", test_df_freq.shape)
print("Columns:", list(test_df_freq.columns))
```

Shape (raw): (6505, 232)

Columns: ['StartServiceA', 'NtGetContextThread', 'IWbemServices_ExecQuery', 'FindResourceA', 'LdrGetProcedureAddress', 'InternetGetConnectedState', 'GetNativeSystemInfo', 'NtOpenProcess', 'CreateToolhelp32Snapshot', 'ShellExecuteExW', 'InternetCrackUrlW', 'CryptExportKey', 'WriteProcessMemory', 'NtCreateFile', 'GlobalMemoryStatus', 'EnumServicesStatusA', 'LdrLoadDll', 'OpenServiceA', 'ReadProcessMemory', 'LookupPrivilegeValueW', 'NtFreeVirtualMemory', 'SetFileTime', 'GetComputerNameA', 'RegDeleteKeyW', 'IsDebuggerPresent', 'LdrUnloadDll', 'OpenSCManagerW', 'GetFileInformationByHandle', 'connect', 'CryptAcquireContextW', 'NtSetValueKey', 'GetSystemTimeAsFileTime', 'InternetConnectA', 'RegSetValueExA', 'DeviceIoControl', 'OpenSCManagerA', 'FindResourceW', 'GetSystemDirectoryA', 'RegCreateKeyExW', 'NtOpenDirectoryObject', 'NtQueryValueKey', 'NtQuerySystemInformation', 'WSAStartup', 'GetFileSize', 'RegDeleteValueW', 'SHGetFolderPathW', 'WSARecv', 'GetSystemMetrics', 'OleInitialize', 'CryptHashData', 'CreateServiceW', 'NtSetContextThread', 'HttpOpenRequestW', 'LdrGetDllHandle', 'GetAsyncKeyState', 'SetUnhandledExceptionFilter', 'RtlAddVectoredExceptionHandler', 'timeGetTime', 'RegQueryInfoKeyW', 'NtCreateMutant', 'GetForegroundWindow', 'FindWindowW', 'FindResourceExW', 'NtReadFile', 'RegOpenKeyExA', 'RegEnumValueW', 'NtEnumerateValueKey', 'CoCreateInstance', 'DnsQuery_A', 'InternetOpenW', 'RemoveDirectoryW', 'CreateJobObjectW', 'socket', 'GetFileAttributesW', 'select', 'InternetQueryOptionA', 'GetFileSizeEx', 'LoadStringA', 'shutdown', 'CryptDecrypt', 'CryptGenKey', 'CopyFileExW', 'NtDeleteKey', 'InternetOpenUrlA', 'CoGetClassObject', 'SetFilePointerEx', 'GlobalMemoryStatusEx', 'DeleteFileW', 'WriteConsoleA', 'GetTimeZoneInformation', 'GetVolumeNameForVolumeMountPointW', 'RtlDecompressBuffer', 'SetFilePointer', 'RegEnumKeyW', 'CopyFileW', 'RegEnumKeyExA', 'CoUninitialize', 'GetAdaptersAddresses', 'RegCloseKey', 'HttpSendRequestA', 'RegEnumKeyExW', 'GetSystemInfo', 'GetBestInterfaceEx', 'getaddrinfo', 'RegOpenKeyExW', 'RegCreateKeyExA', 'NtOpenMutant', 'SetErrorMode', 'CoCreateInstanceEx', 'GetFileType', 'NtOpenFile', 'NtMapViewOfSection', 'GetDiskFreeSpaceExW', 'NtAllocateVirtualMemory', 'NtClose', 'SizeofResource', 'GetFileVersionInfoExW', 'GetKeyState', 'CoInitializeSecurity', 'GetUserNameW', 'GetFileVersionInfoW', 'GetComputerNameW', 'GetFileVersionInfoSizeExW', 'StartServiceW', 'NtOpenKeyEx', 'NtOpenSection', 'NtSetInformationFile', 'SearchPathW', 'CreateDirectoryW', 'InternetSetOptionA', 'LookupAccountSidW', 'GetCursorPos', 'NtTerminateThread', 'getsockname', 'NtQueryInformationFile', 'WriteConsoleW', 'InternetCloseHandle', 'GetSystemWindowsDirectoryW', 'Process32NextW', 'CreateThread', 'RegDeleteValueA', 'recv', 'NtQueryAttributesFile', 'NtUnmapViewOfSection', 'GetFileAttributesExW', 'RegSetValueExW', 'Thread32Next', 'NtOpenKey', 'GetDiskFreeSpaceW', 'EnumWindows', '__exception__', 'Module32FirstW', 'GetVolumePathNameW', 'NtQueryDirectoryFile', 'DrawTextExA', 'CryptProtectData', 'CryptCreateHash', 'UnhookWindowsHookEx', 'CreateServiceA', 'OpenServiceW',

```

'NtProtectVirtualMemory', 'GetAdaptersInfo', 'RemoveDirectoryA',
'HttpQueryInfoA', 'Thread32First', 'DrawTextExW', 'setsockopt',
'GetVolumePathNamesForVolumeNameW', 'RegQueryInfoKeyA', 'CryptEncrypt',
'NtOpenThread', 'InternetCrackUrlA', 'CopyFileA', 'NtQueueApcThread',
'NtCreateSection', 'DeleteUrlCacheEntryA', 'GetSystemDirectoryW',
'RegQueryValueExW', 'LoadStringW', 'RegisterHotKey', 'GetFileVersionInfoSizeW',
'NtWriteFile', 'Module32NextW', 'FindFirstFileExW', 'UuidCreate', 'FindWindowA',
'NtQueryKey', 'GetUserNameA', 'CryptAcquireContextA', 'bind', 'ioctlsocket',
'CreateActCtxW', 'WSASocketA', 'NtDeviceIoControlFile', 'SetWindowsHookExA',
'NtTerminateProcess', 'CreateProcessInternalW', 'NtDuplicateObject',
'ControlService', 'FindWindowExA', 'SetStdHandle', 'NtEnumerateKey',
'NtResumeThread', 'HttpOpenRequestA', 'RegEnumValueA', 'send',
'SetFileAttributesW', 'NtDelayExecution', 'Process32FirstW',
'SHGetSpecialFolderLocation', 'MessageBoxTimeoutA', 'NtCreateKey',
'closesocket', 'SetWindowsHookExW', 'InternetOpenA', 'FindWindowExW',
'NtSuspendThread', 'MoveFileWithProgressW', 'GetSystemWindowsDirectoryA',
'NtCreateThreadEx', 'DeleteUrlCacheEntryW', 'MessageBoxTimeoutW',
'OutputDebugStringA', 'gethostbyname', 'SetEndOfFile', 'InternetConnectW',
'LoadResource', 'CoInitializeEx', 'RegQueryValueExA', 'GetTempPathW',
'GetShortPathNameW', 'FindResourceExA']

```

```
[ ]: test_df_freq.head(10)
```

```

[ ]:      StartServiceA  NtGetContextThread  IWbemServices_ExecQuery  FindResourceA  \
0              0              0              0              1
1              0              0              0              0
2              0              0              0              0
3              0              0              0              0
4              0              0              0              0
5              0              0              0              0
6              0              0              0              0
7              0              0              0             12
8              0              0              0              0
9              0              0              0              0

      LdrGetProcedureAddress  InternetGetConnectedState  GetNativeSystemInfo  \
0              9              0              0
1             11              0              4
2             21              0              1
3             14              0              0
4              5              0              0
5             28              0              0
6              5              0              0
7              8              0              0
8             21              0              0
9             11              0              0

```

	NtOpenProcess	CreateToolhelp32Snapshot	ShellExecuteExW	...	\
0	0	0	0	...	
1	0	0	0	...	
2	0	0	0	...	
3	0	0	0	...	
4	0	0	0	...	
5	0	0	0	...	
6	0	0	0	...	
7	0	0	0	...	
8	0	0	0	...	
9	0	0	0	...	

	OutputDebugStringA	gethostbyname	SetEndOfFile	InternetConnectW	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	9	0	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	0	0	0	
9	0	0	0	0	

	LoadResource	CoInitializeEx	RegQueryValueExA	GetTempPathW	\
0	23	0	0	0	
1	0	1	0	0	
2	8	0	0	0	
3	0	1	1	0	
4	1	2	0	0	
5	0	0	1	0	
6	1	2	0	0	
7	12	0	0	0	
8	0	1	1	0	
9	1	2	0	0	

	GetShortPathNameW	FindResourceExA
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	13
8	0	0
9	0	0

[10 rows x 232 columns]

Q: Can you use the test vocabulary to build the new test dataframe? If not, how do you handle API calls in the test set that do not exist in the training vocabulary? No, you cannot use the test vocabulary to build the test dataframe.

Machine learning models require the feature space (columns) during inference (testing) to be identical to the feature space used during training.

Common strategies for handling API calls in the test set that did not exist in the training vocabulary include:

1. **Add an “Unknown” Column:** Map all unseen API calls to a single generic token (e.g., <UNK>) or a specific “is_unknown” feature. This preserves the information that *an* action occurred, even if the specific action is unrecognized.
2. **Ignore Them:** Simply discard or zero-out any API calls that were not seen during training.

Decision: In this specific case, since there are **very few unseen API calls** (i.e. *only 3 samples*) in the test set, we decided to **ignore them**. This approach simplifies the pipeline without significantly impacting the model’s performance.

Sparse Vectors Analysis

```
[ ]: # --- Non-zero elements analysis for training data ---

# Calculate average non-zero elements per row
avg_nonzero_train = (train_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in train_df_freq: {avg_nonzero_train:.2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_train = avg_nonzero_train / train_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in train_df_freq: {ratio_nonzero_train:.4f}")
```

Average non-zero elements per row in train_df_freq: 21.95

Ratio of non-zero elements per row in train_df_freq: 0.0851

```
[ ]: # --- Non-zero elements analysis for test data ---

# Calculate average non-zero elements per row
avg_nonzero_test = (test_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in test_df_freq: {avg_nonzero_test:.2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_test = avg_nonzero_test / test_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in test_df_freq: {ratio_nonzero_test:.4f}")
```

Average non-zero elements per row in test_df_freq: 24.28
Ratio of non-zero elements per row in test_df_freq: 0.1046

Q: How many non-zero elements per row do you have on average in the training set? How many in the test set? What is the ratio with respect to the number of elements per row? On average:

- **Training set:** ~22 non-zero elements per row, which is **8.5%** of all elements.
- **Test set:** ~24 non-zero elements per row, which is **10.5%** of all elements.

Q: The original API sequences were ordered. Is it still the case now in the frequency-based dataframe? Why? No, the original order of API calls is **not preserved** in the frequency-based dataframe.

This is because the frequency-based representation **counts how many times each API call occurs** in a sequence, without keeping track of their positions. Each row now simply contains **one column per API call in the vocabulary**, with the value representing its count, so the sequential order is lost.

This approach converts sequences into fixed-dimensional vectors, simplifying the computation but discarding temporal information. As a result, complex behavioral patterns based on the chronology of calls cannot be captured by this representation.

1.2.4 Feed the frequency-based datasets to a classifier

- Any classifier can be used (shallow/deep neural or non-neural).
- Goal: evaluate baseline performance on sparse vectors without sequence information.

Training

```
[ ]: # --- Training with a classifier ---

# Split data into features and target
X = train_df_freq
y = df_train['is_malware']

# Create a validation set from training set
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

# Construct test set with features aligned to train_vocab
# TODO: justify this strategy in the report (too few samples)
X_test = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for seq in
    df_test['api_call_sequence']])
y_test = df_test['is_malware']

# Train a simple classifier (using default hyperparameters for a simple
    baseline)
```

```

clf = LogisticRegression(random_state=42, solver='liblinear') # 'liblinear'
    ↪ solver is good for small datasets and sparse data
clf.fit(X_train, y_train)

# Compute training and validation loss
y_train_proba = clf.predict_proba(X_train)
y_val_proba = clf.predict_proba(X_val)

print(f"Training log-loss: {log_loss(y_train, y_train_proba):.4f}")
print(f"Validation log-loss: {log_loss(y_val, y_val_proba):.4f}")

```

Training log-loss: 0.0806
Validation log-loss: 0.1224

Evaluation

```

[ ]: def evaluate_model(model, X, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained scikit-learn model on a given dataset and return the
    ↪ classification report.
    Handles missing predicted classes gracefully (zero_division=0).
    """
    # Get predictions
    y_pred = model.predict(X)

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:
    ↪ {missing_classes}")

    # Generate classification report
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report

```

```

[ ]: def plot_cm_task1(model, X_test, y_test, model_name, save_dir=None):
    """
    Specific for Task 1 (Scikit-Learn models).
    Generates predictions using .predict() and then plots.
    """
    # 1. Get predictions (Scikit-learn models work directly on X_test)
    y_pred = model.predict(X_test)

    # 2. Use the core helper to plot
    # Note: y_test might need to be converted to a simple list/array if it's a
    ↪ pandas Series

```

```
_internal_plot_cm(y_test, y_pred, model_name, save_dir)
```

```
[ ]: # --- Evaluate validation and test set and print classification reports ---

print(f"\nValidation classification report:\n")
report = evaluate_model(clf, X_val, y_val, "Logistic Regression (Validation)")
print(report)

print(f"\nTest classification report:\n")
report = evaluate_model(clf, X_test, y_test, "Logistic Regression (Test)")
print(report)
```

Validation classification report:

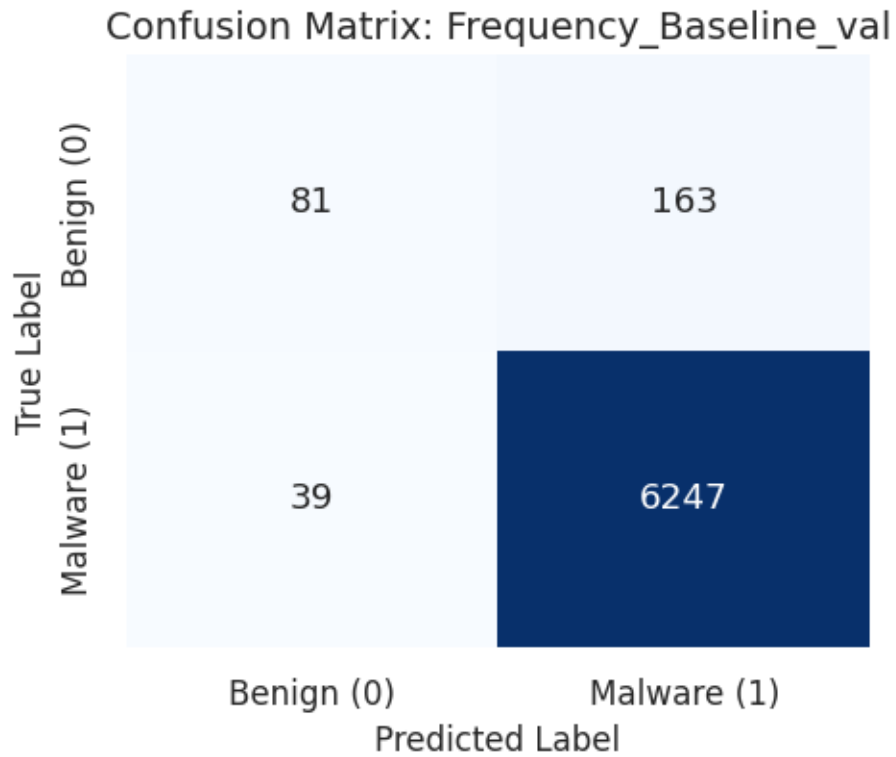
	precision	recall	f1-score	support
0	0.6750	0.3320	0.4451	244
1	0.9746	0.9938	0.9841	6286
accuracy			0.9691	6530
macro avg	0.8248	0.6629	0.7146	6530
weighted avg	0.9634	0.9691	0.9639	6530

Test classification report:

	precision	recall	f1-score	support
0	0.6885	0.3457	0.4603	243
1	0.9751	0.9939	0.9844	6262
accuracy			0.9697	6505
macro avg	0.8318	0.6698	0.7223	6505
weighted avg	0.9644	0.9697	0.9648	6505

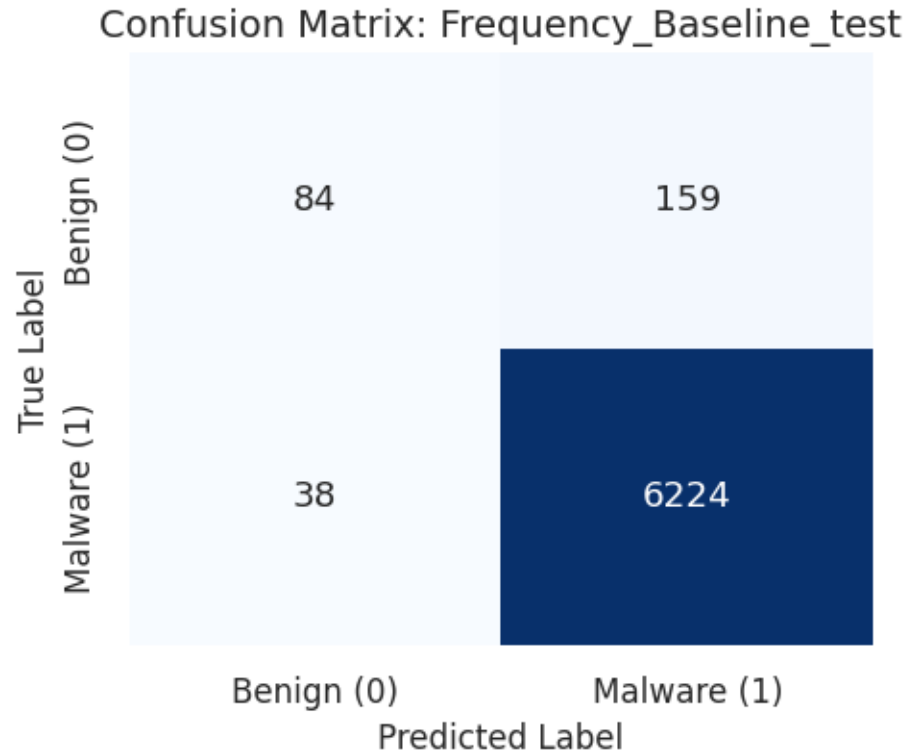
```
[ ]: plot_cm_task1(
    model=clf,
    X_test=X_val,
    y_test=y_val,
    model_name="Frequency_Baseline_val",
    save_dir=results_path + 'images/task1_plots/'
)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_val.png



```
[ ]: plot_cm_task1(  
    model=clf,  
    X_test=X_test,  
    y_test=y_test,  
    model_name="Frequency_Baseline_test",  
    save_dir=results_path + 'images/task1_plots/'  
)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task1_plots/cm_frequency_baseline_test.png



Q: Report how you chose the hyperparameters of your classifier, and the final performance on the test set. In Machine Learning problems, it is always good practice to compare against baseline solutions.

We used a simple logistic regression as a baseline classifier. The hyperparameters were mostly left at their defaults. We specifically chose:

- `solver='liblinear'` → suitable for small and sparse datasets
- `random_state=42` → ensures reproducibility.

No additional hyperparameter tuning (like regularization strength C) was performed, since this was intended as a simple baseline.

Final performance on the test set: - the classifier strongly predicts the malware class, but is weaker on the minority class (benign samples).

Q: Is the final performance good, even ignoring the order of API calls and handling very sparse vectors? Yes, the performance is **good for a baseline frequency-based model**, given that: - The **order of API calls is ignored**, so any sequential patterns are lost. - The feature vectors are **very sparse** (only ~8-10% non-zero elements per row).

Despite these limitations, the model achieves **high accuracy and excellent detection of malware**, which indicates that the presence/frequency of certain API calls alone is highly informative for this classification task.